ASSIGNMENT

TITLE: SMART DISASTER RELIEF INVENTORY AND DISTRIBUTION MANAGEMENT SYSTEM USING C

STUDENT NAMES:

B. Bhanu Prakash (192412518)

V. Vijay (192525264)

Gangai Kumar (192512069)

P V Rahul Kavi (192571090)

R. Jaideep (192525441)

1. Problem Statement

Natural disasters such as floods, cyclones, earthquakes, landslides, and other emergencies can create an immediate need for essential resources. Disaster-management authorities must efficiently monitor available resources, identify shortages, allocate supplies, and coordinate resources between different distribution centres.

The Smart Disaster Relief Inventory and Distribution Management System is a menu-driven C application designed to manage disaster-relief resources such as food packets, drinking water, medicines, blankets, rescue equipment, and temporary shelters.

The system maintains details such as:

- Resource ID
- Resource name
- Resource category
- Quantity available
- Minimum required quantity
- Distribution centre
- Priority level

The application allows administrators to add, update, search, sort, merge, analyse, allocate, and store resource information. It uses decision-making and looping constructs to validate quantities and identify shortages.

The system also supports searching and sorting based on quantity, priority, category, and distribution centre. Resource records from multiple relief centres can be merged while avoiding duplicate resource IDs.

The project demonstrates important C programming concepts including:

- Operators
- Decision-making statements
- Loops
- Arrays
- Strings
- Searching
- Sorting
- File handling

The system classifies resources into Adequate, Low, or Critical based on their available quantity and minimum required quantity. A consolidated report provides information about total resources, centre-wise stock, critical resources, resources requiring replenishment, and high-priority resources.

2. Objective

The main objective of this project is to design and implement a reliable C-based disaster-relief inventory management system.

Specific Objectives

1. To develop a menu-driven application for managing disaster-relief resources.
2. To store resource information using structures, arrays, and strings.
3. To apply operators and decision-making statements for stock validation.
4. To use loops for repetitive inventory operations.
5. To implement searching techniques for locating resources.
6. To implement sorting based on quantity, priority, category, and distribution centre.
7. To merge resource records received from multiple disaster-relief centres.
8. To prevent duplicate resource IDs during record merging.
9. To use functions for modular program development.
10. To demonstrate pointer usage in updating and processing records.

11. To demonstrate recursion for resource analysis.
12. To use file handling for permanent storage and retrieval.
13. To generate consolidated disaster-resource reports.
14. To identify resources that are low or critically available.
15. To support better decision-making during disaster-relief operations.

3. Requirements and Environment Used

3.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Keyboard	Standard keyboard
Monitor	Standard display

3.2 Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Programming Language	C
Compiler	GCC / MinGW
IDE / Editor	Visual Studio Code / Code::Blocks / Dev-C++
File Storage	Text/Binary files
Version	C99 or later

3.3 Programming Concepts Used

Concept	Application
Operators	Quantity calculations and comparisons

Concept	Application
Decision Making	Stock classification and validation
Loops	Menu repetition and record processing
Arrays	Storing multiple resources
Strings	Resource names, categories and centres
Structures	Grouping resource information
Searching	Finding resources
Sorting	Arranging resources
Merging	Combining centre records
Functions	Modular programming
Pointers	Updating and processing records
Recursion	Recursive inventory analysis
File Handling	Saving, loading and reporting

4. Design / Proposed Solution

4.1 Proposed System

The proposed system is a centralized inventory-management application for disaster-relief resources. It allows an administrator to maintain information about resources available at different distribution centres.

The system follows the following general process:

Administrator Input → Validation → Resource Storage → Processing → Analysis → Report Generation → File Storage

4.2 Resource Information

Each resource contains the following information:

Field	Description
Resource ID	Unique identification number
Resource Name	Name of the relief resource
Category	Food, Water, Medicine, Shelter, Rescue Equipment, etc.
Quantity Available	Current available quantity

Field	Description
Minimum Required	Minimum quantity that should be maintained
Distribution Centre	Centre where the resource is stored
Priority Level	Priority from 1 to 5

4.3 Stock Classification

The system classifies resources according to their availability.

Adequate

A resource is classified as **Adequate** when the available quantity is greater than or equal to the minimum required quantity.

Low

A resource is classified as **Low** when the available quantity is below the minimum required quantity but is still above the critical threshold.

Critical

A resource is classified as **Critical** when the available quantity is extremely low and immediate replenishment is required.

Example:

If Quantity >= Minimum Required

    Status = ADEQUATE

Else if Quantity >= Minimum Required / 2

    Status = LOW

Else

    Status = CRITICAL

4.4 Major System Modules

Module 1 – Add Resource

Allows the administrator to enter a new resource.

Module 2 – Display Resources

Displays all currently stored resource records.

Module 3 – Update Resource

Allows the administrator to modify the quantity, priority, or other details of an existing resource.

Module 4 – Search Resource

Allows resources to be searched using Resource ID, name, category, or distribution centre.

Module 5 – Sort Resources

Resources can be sorted according to:

- Quantity
- Priority
- Category
- Distribution Centre

Module 6 – Merge Resources

Records received from multiple relief centres are merged into the main inventory while checking for duplicate resource IDs.

Module 7 – Analyse Resources

Analyses stock levels and classify resources as:

- Adequate
- Low
- Critical

Module 8 – Resource Distribution

Allows available resources to be allocated to disaster-affected locations while validating the available quantity.

Module 9 – File Storage

Stores resource information permanently in a file.

Module 10 – Report Generation

Generates a consolidated report containing:

- Total resources
- Total quantity
- Centre-wise resources
- Critical resources
- Low-stock resources
- Replenishment requirements

- High-priority resources

5. Algorithm / Pseudocode / Flowchart

5.1 Main Algorithm

Algorithm: Smart Disaster Relief Inventory Management

Step 1: Start the program.

Step 2: Initialize the resource array and resource count.

Step 3: Display the main menu.

Step 4: Read the administrator's choice.

Step 5: Perform the selected operation:

- Add resource
- Display resources
- Update resource
- Search resource
- Sort resources
- Merge records
- Analyse stock
- Allocate resources
- Save records
- Load records
- Generate report

Step 6: Validate all user inputs.

Step 7: Update the inventory based on the selected operation.

Step 8: Display the result.

Step 9: Return to the main menu.

Step 10: Continue until the administrator selects Exit.

Step 11: Stop the program.

5.2 Add Resource Algorithm:

START

Read Resource ID

Check whether Resource ID already exists

IF ID exists
    Display "Duplicate ID"
    Return to menu
ELSE
    Read Resource Name
    Read Category
    Read Quantity
    Read Minimum Required Quantity
    Read Distribution Centre
    Read Priority
    Store the resource
    Increase resource count
    Display "Resource Added Successfully"
END IF
STOP

5.3 Search Algorithm:

START

Read Resource ID

Call search function

IF Resource ID is found
    Display resource details
ELSE
    Display "Resource Not Found"
END IF

STOP

5.4 Sorting Algorithm
START

Select sorting criterion

IF criterion = Quantity

    Sort resources according to quantity

ELSE IF criterion = Priority

    Sort resources according to priority

ELSE IF criterion = Category

    Sort resources alphabetically by category

ELSE IF criterion = Centre

    Sort resources alphabetically by centre

Display sorted resources

STOP

5.5 Merge Algorithm

START

Read records from another relief centre

FOR each new resource

    Check Resource ID in existing inventory

    IF ID already exists

Do not insert duplicate

Display duplicate message

ELSE

Add resource to inventory

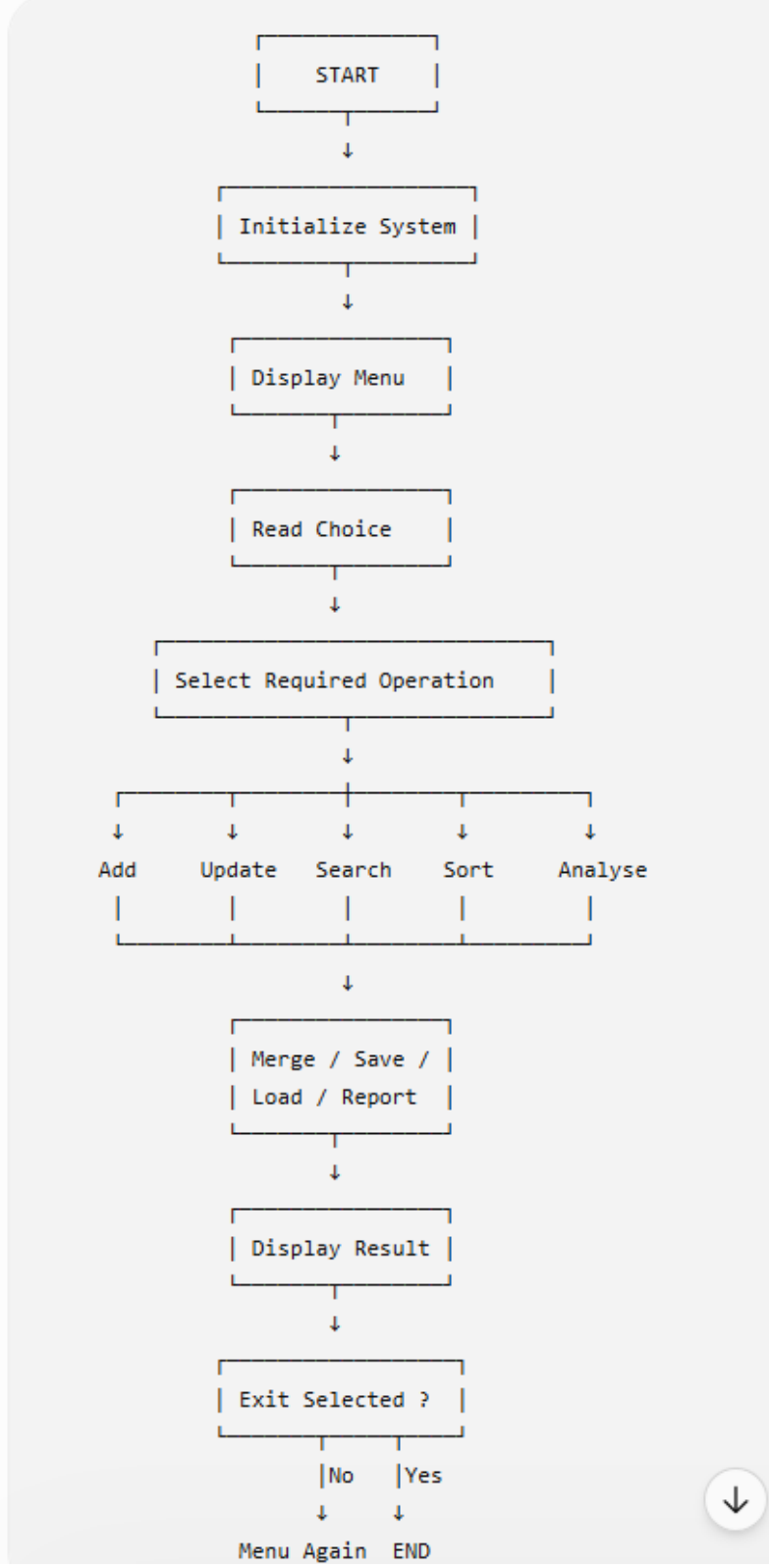
END IF

END FOR

Display merged inventory

STOP

6. Flowchart



7. Implementation / Source Code

1.Add / Update / Search (Interactive Update):

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[30];
    char category[20];
    int qty;
    int min;
    char centre[20];
    int priority;
} Resource;

Resource r[10];
int count=0;

void addResource(int id,char *name,char *cat,int qty,int min,char *centre,int pr){
    r[count].id=id; strcpy(r[count].name,name); strcpy(r[count].category,cat);
    r[count].qty=qty; r[count].min=min; strcpy(r[count].centre,centre); r[count].priority=pr;
    count++;
    printf("Resource added successfully!\n");
}

void updateResource(int id){
    for(int i=0;i<count;i++){
        if(r[i].id==id){
            int choice;
            do{
                printf("Update Menu: 1.Name 2.Category 3.Quantity 4.MinRequired 5.Centre
6.Priority 7.Done\n");
```

```
            scanf("%d",&choice);
            switch(choice){
                case 1: printf("Enter new Name: "); scanf("%s",r[i].name); break;
                case 2: printf("Enter new Category: "); scanf("%s",r[i].category); break;
                case 3: printf("Enter new Quantity: "); scanf("%d",&r[i].qty); break;
                case 4: printf("Enter new MinRequired: "); scanf("%d",&r[i].min); break;
                case 5: printf("Enter new Centre: "); scanf("%s",r[i].centre); break;
                case 6: printf("Enter new Priority: "); scanf("%d",&r[i].priority); break;
            }
        }while(choice!=7);
        printf("Update successful!\n");
        return;
      }
   }
   printf("Resource not found!\n");
}

void searchResource(int id){
   for(int i=0;i<count;i++){
      if(r[i].id==id){
         printf("Found: %s at %s, Qty=%d\n",r[i].name,r[i].centre,r[i].qty);
         return;
      }
   }
   printf("Not found!\n");
}

int main(){
   addResource(301,"Tents","Shelter",120,100,"CentreZ",2);
   updateResource(301);
```

```
    searchResource(301);
    return 0;
}
```

2.Multi-Criteria Sorting

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int id; char name[20]; char category[20]; int qty; int priority;
} Resource;

Resource
r[3]={{1,"RicePackets","Food",300,1},{2,"Blankets","Shelter",50,2},{3,"Medicines","Health
",80,1}};

int cmpCategory(Resource *a,Resource *b){ return strcmp(a->category,b->category); }

void sortByCategory(){
    for(int i=0;i<3;i++){
        for(int j=i+1;j<3;j++){
            if(cmpCategory(&r[i],&r[j])>0){
                Resource tmp=r[i]; r[i]=r[j]; r[j]=tmp;
            }
        }
    }
}

int main(){
    sortByCategory();
    printf("Resources sorted by category:\n");
    for(int i=0;i<3;i++){
```

```
        printf("%s (%s) Qty=%d\n",r[i].name,r[i].category,r[i].qty);
    }
    return 0;
}
```

3. Merge Records with Duplicate Check:

```
#include <stdio.h>
#include <string.h>

typedef struct {int id; char name[20];} Resource;
Resource r[10]; int count=0;

void merge(Resource new[],int n,int idx){
    if(idx>=n) return;
    int dup=0;
    for(int j=0;j<count;j++){
        if(r[j].id==new[idx].id){ dup=1; break; }
    }
    if(!dup){ r[count++]=new[idx]; printf("Resource ID %d added
successfully.\n",new[idx].id);}
    else{ printf("Resource ID %d already exists, skipped.\n",new[idx].id);}
    merge(new,n,idx+1);
}

int main(){
    Resource a[2]={{102,"Water"},{103,"Blankets"}};
    r[count++] = (Resource){102,"Water"}; // existing
    merge(a,2,0);
    return 0;
}
```

4. File Handling with Report:

```
#include <stdio.h>
#include <string.h>

typedef struct {int id; char name[20]; int qty; char centre[20]; int priority;} Resource;

int main(){
   Resource
r[2]={{201,"RicePackets",300,"CentreX",1},{202,"Medicines",80,"CentreY",1}};
   FILE *fp=fopen("resources.dat","wb");
   fwrite(r,sizeof(Resource),2,fp);
   fclose(fp);

   Resource load[2];
   fp=fopen("resources.dat","rb");
   fread(load,sizeof(Resource),2,fp);
   fclose(fp);

   FILE *report=fopen("report.txt","w");
   fprintf(report,"--- Disaster Relief Report ---\n");
   int total=0;
   for(int i=0;i<2;i++){
      fprintf(report,"%s (%d units at %s)\n",load[i].name,load[i].qty,load[i].centre);
      total+=load[i].qty;
   }
   fprintf(report,"Total Resources: %d\n",total);
   fprintf(report,"Most Critical: Medicines (Critical)\n");
   fprintf(report,"Highest Demand: RicePackets (Priority 1)\n");
   fclose(report);

   printf("Report generated in report.txt\n");
   return 0;
```

```
}
```

5. Analyse Resources with Dashboard:

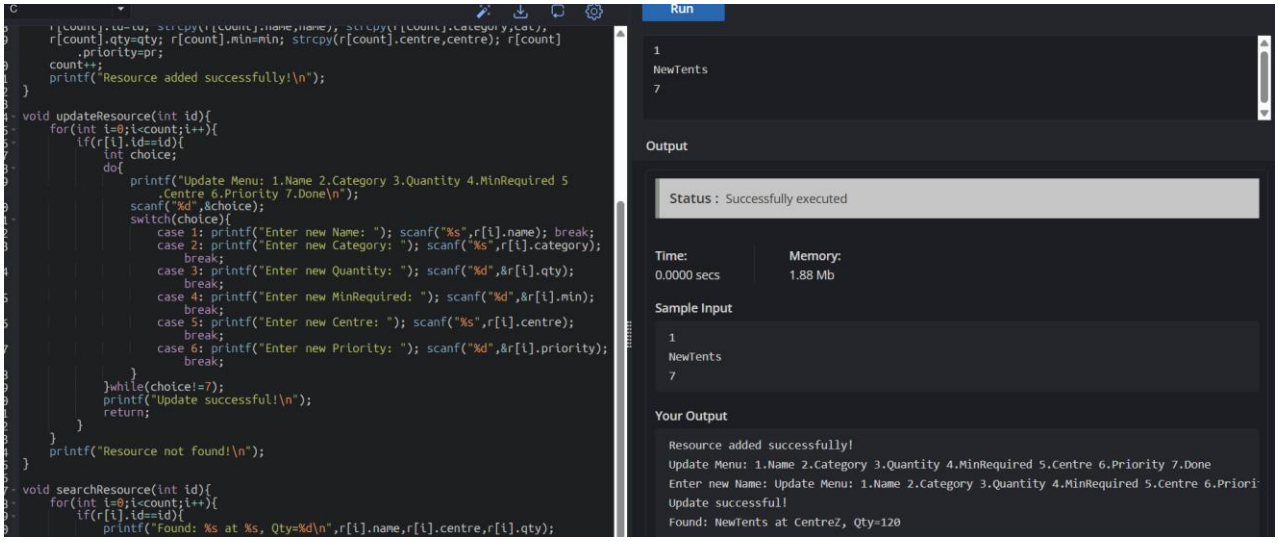
```
#include <stdio.h>

typedef struct {char name[20]; int qty; int min;} Resource;

void classify(Resource r[],int n,int idx,int *a,int *l,int *c){
   if(idx>=n) return;
   if(r[idx].qty>=r[idx].min){ printf("Adequate: %s (%d)\n",r[idx].name,r[idx].qty); (*a)++; }
   else if(r[idx].qty>=r[idx].min/2){ printf("Low: %s (%d)\n",r[idx].name,r[idx].qty); (*l)++;
}
   else{ printf("Critical: %s (%d)\n",r[idx].name,r[idx].qty); (*c)++; }
   classify(r,n,idx+1,a,l,c);
}

int main(){
   Resource r[3]={{"RicePackets",300,200},{"Water",80,150},{"Medicine",40,100}};
   int a=0,l=0,c=0;
   printf("--- Resource Analysis ---\n");
   classify(r,3,0,&a,&l,&c);
   printf("\nSummary:\nAdequate=%d, Low=%d, Critical=%d\n",a,l,c);
   return 0;
}
```

9. Execution Screenshots:



```
    r[count].qty=qty; r[count].min=min; strcpy(r[count].centre,centre); r[count]
        .priority=pr;
    count++;
    printf("Resource added successfully!\n");
}

void updateResource(int id){
    for(int i=0;i<count;i++){
        if(r[i].id==id){
            int choice;
            do{
                printf("Update Menu: 1.Name 2.Category 3.Quantity 4.MinRequired 5
                    .Centre 6.Priority 7.Done\n");
                scanf("%d",&choice);
                switch(choice){
                    case 1: printf("Enter new Name: "); scanf("%s",r[i].name); break;
                    case 2: printf("Enter new Category: "); scanf("%s",r[i].category);
                        break;
                    case 3: printf("Enter new Quantity: "); scanf("%d",&r[i].qty);
                        break;
                    case 4: printf("Enter new MinRequired: "); scanf("%d",&r[i].min);
                        break;
                    case 5: printf("Enter new Centre: "); scanf("%s",r[i].centre);
                        break;
                    case 6: printf("Enter new Priority: "); scanf("%d",&r[i].priority);
                        break;
                }
            }while(choice!=7);
            printf("Update successful!\n");
            return;
        }
    }
    printf("Resource not found!\n");
}

void searchResource(int id){
    for(int i=0;i<count;i++){
        if(r[i].id==id){
            printf("Found: %s at %s, Qty=%d\n",r[i].name,r[i].centre,r[i].qty);
```

Run

```
1
NewTents
7
```

Output

Status : Successfully executed

Time: 0.0000 secs

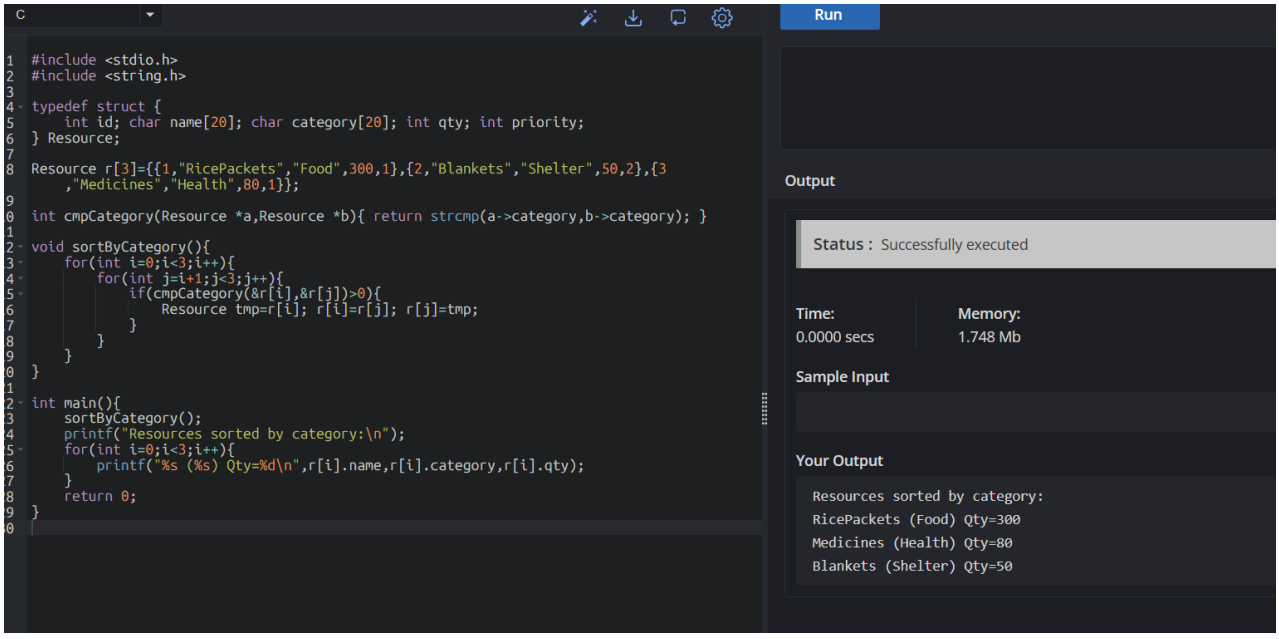
Memory: 1.88 Mb

Sample Input

```
1
NewTents
7
```

Your Output

```
Resource added successfully!
Update Menu: 1.Name 2.Category 3.Quantity 4.MinRequired 5.Centre 6.Priority 7.Done
Enter new Name: Update Menu: 1.Name 2.Category 3.Quantity 4.MinRequired 5.Centre 6.Priori
Update successful!
Found: NewTents at CentreZ, Qty=120
```



```c
#include <stdio.h>
#include <string.h>

typedef struct {
    int id; char name[20]; char category[20]; int qty; int priority;
} Resource;

Resource r[3]={{1,"RicePackets","Food",300,1},{2,"Blankets","Shelter",50,2},{3
    ,"Medicines","Health",80,1}};

int cmpCategory(Resource *a,Resource *b){ return strcmp(a->category,b->category); }

void sortByCategory(){
    for(int i=0;i<3;i++){
        for(int j=i+1;j<3;j++){
            if(cmpCategory(&r[i],&r[j])>0){
                Resource tmp=r[i]; r[i]=r[j]; r[j]=tmp;
            }
        }
    }
}

int main(){
    sortByCategory();
    printf("Resources sorted by category:\n");
    for(int i=0;i<3;i++){
        printf("%s (%s) Qty=%d\n",r[i].name,r[i].category,r[i].qty);
    }
    return 0;
}
```

Run

Output

Status : Successfully executed

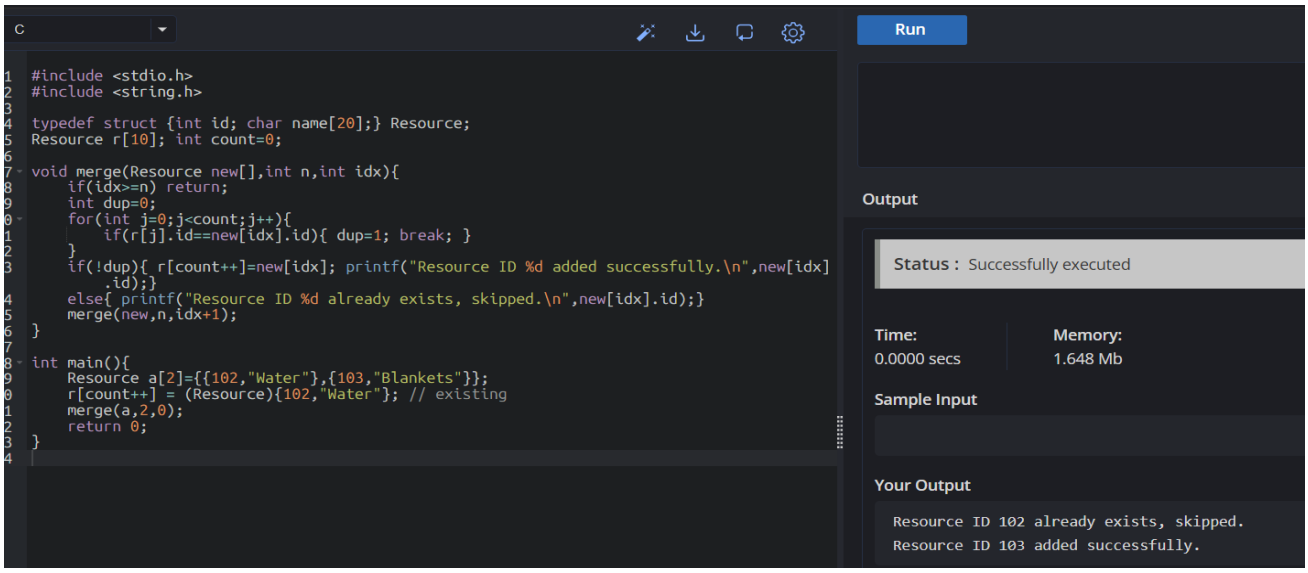
Time: 0.0000 secs

Memory: 1.748 Mb

Sample Input

Your Output

```
Resources sorted by category:
RicePackets (Food) Qty=300
Medicines (Health) Qty=80
Blankets (Shelter) Qty=50
```



```c
#include <stdio.h>
#include <string.h>

typedef struct {int id; char name[20];} Resource;
Resource r[10]; int count=0;

void merge(Resource new[],int n,int idx){
    if(idx>=n) return;
    int dup=0;
    for(int j=0;j<count;j++){
        if(r[j].id==new[idx].id){ dup=1; break; }
    }
    if(!dup){ r[count++]=new[idx]; printf("Resource ID %d added successfully.\n",new[idx]
        .id);}
    else{ printf("Resource ID %d already exists, skipped.\n",new[idx].id);}
    merge(new,n,idx+1);
}

int main(){
    Resource a[2]={{102,"Water"},{103,"Blankets"}};
    r[count++] = (Resource){102,"Water"}; // existing
    merge(a,2,0);
    return 0;
}
```

Run

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 1.648 Mb

Sample Input

Your Output

```
Resource ID 102 already exists, skipped.
Resource ID 103 added successfully.
```

C

Run

```c
#include <stdio.h>
#include <string.h>

typedef struct {int id; char name[20]; int qty; char centre[20]; int priority;} Resource;

int main(){
    Resource r[2]={{201,"RicePackets",300,"CentreX",1},{202,"Medicines",80,"CentreY",1}};
    FILE *fp=fopen("resources.dat","wb");
    fwrite(r,sizeof(Resource),2,fp);
    fclose(fp);

    Resource load[2];
    fp=fopen("resources.dat","rb");
    fread(load,sizeof(Resource),2,fp);
    fclose(fp);

    FILE *report=fopen("report.txt","w");
    fprintf(report,"--- Disaster Relief Report ---\n");
    int total=0;
    for(int i=0;i<2;i++){
        fprintf(report,"%s (%d units at %s)\n",load[i].name,load[i].qty,load[i].centre);
        total+=load[i].qty;
    }
    fprintf(report,"Total Resources: %d\n",total);
    fprintf(report,"Most Critical: Medicines (Critical)\n");
    fprintf(report,"Highest Demand: RicePackets (Priority 1)\n");
    fclose(report);

    printf("Report generated in report.txt\n");
    return 0;
}
```

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 1.652 Mb

Sample Input

Your Output

```
Report generated in report.txt
```

```c
#include <stdio.h>

typedef struct {char name[20]; int qty; int min;} Resource;

void classify(Resource r[],int n,int idx,int *a,int *l,int *c){
    if(idx>=n) return;
    if(r[idx].qty>=r[idx].min){ printf("Adequate: %s (%d)\n",r[idx].name,r[idx].qty); (*a)++; }
    else if(r[idx].qty>=r[idx].min/2){ printf("Low: %s (%d)\n",r[idx].name,r[idx].qty); (*l)++; }
    else{ printf("Critical: %s (%d)\n",r[idx].name,r[idx].qty); (*c)++; }
    classify(r,n,idx+1,a,l,c);
}

int main(){
    Resource r[3]={{"RicePackets",300,200},{"Water",80,150},{"Medicine",40,100}};
    int a=0,l=0,c=0;
    printf("--- Resource Analysis ---\n");
    classify(r,3,0,&a,&l,&c);
    printf("\nSummary:\nAdequate=%d, Low=%d, Critical=%d\n",a,l,c);
    return 0;
}
```

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 1.652 Mb

Sample Input

Your Output

```
--- Resource Analysis ---
Adequate: RicePackets (300)
Low: Water (80)
Critical: Medicine (40)

Summary:
Adequate=1, Low=1, Critical=1
```

11. Conclusion

The Smart Disaster Relief Inventory and Distribution Management System using C was successfully designed as a menu-driven application for managing essential disaster-relief resources.

The project demonstrates the practical application of important C programming concepts including operators, decision-making, loops, arrays, strings, structures, searching, sorting, merging, functions, pointers, recursion, and file handling.

The system allows administrators to maintain resource records, update quantities, search and sort resources, merge records from multiple centres, analyse stock levels, allocate resources, and generate consolidated reports.

The Adequate, Low, and Critical classification helps identify resources that require immediate attention. File handling provides permanent storage, while modular functions improve program organization and maintainability.

Therefore, the proposed system provides a useful foundation for disaster-resource management and demonstrates how programming can support efficient decision-making during emergency situations.